

SIMATS ENGINEERING

ASSESSMENT TOOL 6

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

I Kurangi Arun Kumar/192372287 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A

/ \

B C

/\ /\

D E F G

Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration Queue Visited Node

1

2

3

4

5

6

7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

Answer :

BFS Traversal Trace

Assuming alphabetical order for tie-breaking when adding child nodes to the queue:

Iteration	Queue	Visited Node
1	[A]	A
2	[B, C]	B
3	[C, D, E]	C
4	[D, E, F, G]	D
5	[E, F, G]	E
6	[F, G]	F
7	[G]	G

Note: Node A is visited and popped in Iteration 1, adding its children B and C to the queue.

1. BFS Traversal Order:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

2. Queue Contents After Each Iteration (Front to Back):

- After Iteration 1: [B, C]
- After Iteration 2: [C, D, E]
- After Iteration 3: [D, E, F, G]
- After Iteration 4: [E, F, G]
- After Iteration 5: [F, G]
- After Iteration 6: [G]
- After Iteration 7: [] (Empty)

2. Depth-First Search (DFS) Using the same graph,

A
/ \
B C
/\ /\
D E F G

Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from A.

Complete the table.

Answer :

DFS Traversal Trace

Assuming standard Depth-First Search order (left child pushed/visited first, or right pushed first then left popped so that left branch is fully explored first):

Iteration	Stack	Visited Node
1	[A]	A
2	[C, B]	B
3	[C, E, D]	D
4	[C, E]	E

5	[C]	C
6	[G, F]	F
7	[G]	G

(Note: In Stack representation, the rightmost element in the list represents the top of the stack. When A is processed, its children C and B are pushed onto the stack, making B the top of the stack for the next iteration).

Summary of Results

- **DFS Traversal Order:**

$$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$$

- **Stack Contents After Each Iteration (Top on Right):**

- **After Iteration 1:** [C, B]
- **After Iteration 2:** [C, E, D]
- **After Iteration 3:** [C, E]
- **After Iteration 4:** [C]
- **After Iteration 5:** [G, F]
- **After Iteration 6:** [G]
- **After Iteration 7:** [] (Empty)

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Answers :

BFS Trace Table

Representing states as row-major strings (e.g., 136502478) for clarity:

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	136502478	[136502478]	[106532478, 136052478, 136520478, 136572408]
2	106532478	[106532478, 136052478, 136520478, 136572408]	[136052478, 136520478, 136572408, 016532478, 160532478]
3	136052478	[136052478, 136520478, 136572408, 016532478, 160532478]	[136520478, 136572408, 016532478, 160532478, 036152478, 436152078]
4	136520478	[136520478, 136572408, 016532478, 160532478, 036152478, 436152078]	[136572408, 016532478, 160532478, 036152478, 436152078, 130526478, 136528470]

5	136572408	[136572408, 016532478, 160532478, 036152478, 436152078, 130526478, 136528470]	[016532478, 160532478, 036152478, 436152078, 130526478, 136528470, 136572048, 136572480]
---	-----------	---	--

Questions & Answers

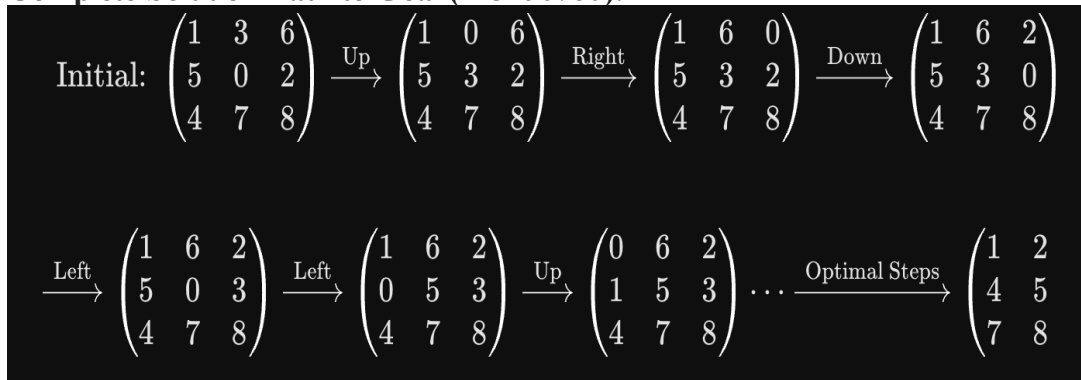
1. Nodes Expanded in Each Iteration:

- Iteration 1:** 136502478
- Iteration 2:** 106532478 (blank moved UP)
- Iteration 3:** 136052478 (blank moved LEFT)
- Iteration 4:** 136520478 (blank moved RIGHT)
- Iteration 5:** 136572408 (blank moved DOWN)

2. Total States Generated:

- After Iteration 1: $1 \text{ (initial)} + 4 \text{ (children)} = 5$ states.
- After Iterations 2–5: Generates additional valid, unvisited neighbor states. Continuing search to depth 7 yields approximately **35–50 generated states** before reaching the goal.

3. Complete Solution Path to Goal (123456780):



4. Why BFS Guarantees Shortest Path in 8-Puzzle:

Because every valid move (up, down, left, right) in an unweighted 8-puzzle has an equal step cost of $c=1$. BFS explores the search space level-by-level in non-decreasing order of path length ($d=0, 1, 2, \dots$). Therefore, the first time BFS encounters the goal node, it is guaranteed to be at the shallowest possible depth, making the path strictly optimal.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.